

`</>` JAVASCRIPT COMPLETE COURSE — DAY 2

Data Types, Conversions & Operators

Mastering the core foundational building blocks of modern JavaScript development.

FORMAT

Concept → Live Code → Practice

TARGET LENGTH

30 Minutes

Today's Session Roadmap

01 Primitive Data Types (String, Number, Boolean, Null, Undefined, Symbol, BigInt)

02 Non-Primitive Reference Types & typeof Operator Quirk

03 Type Conversion (Explicit) vs Type Coercion (Implicit)

04 Falsy Values & Strict (===) vs Loose (==) Equality

05 JavaScript Operators & Hands-on Practice Task

Where We Left Off & What's Next

🔄 Session 1 Quick Recap

- ✓ **HTML Integration:** Linking JS via script tags
- ✓ **Developer Tools:** Inspecting console.log outputs
- ✓ **Variables:** Differences between var, let, and const

⚡ Today's Core Focus

Variables hold values — but **what are those values?**

Understanding how JS classifies and transforms data is the backbone of writing bug-free logic.

What are Data Types in JavaScript?



The Wardrobe Analogy

Different drawers hold clothes, shoes, or books. Similarly, JS stores data in specific categories based on its nature and behavior.

Primitive Data Types

Simple, immutable, single values stored directly in memory locations.

7 Types: String, Number, Boolean, Undefined, Null, Symbol, BigInt

Non-Primitive (Reference)

Complex structures holding collections of values or references to memory.

Key Collections: Objects, Arrays, Functions

Primitive Types: String, Number, Boolean

```
// 1. String – Text wrapped in quotes  
let name = "Ali Khan";  
let greeting = 'Hello World';  
let sentence = `My name is ${name}`; // Template literal  
  
// 2. Number – Integers & Decimals combined  
let age = 25;  
let price = 99.99;  
  
// 3. Boolean – True / False decisions  
let isLoggedIn = true;
```

A Strings

Can use single quotes, double quotes, or backticks for template literals.

Numbers

No separate float or int types — JS represents all numbers uniformly.

🔵 Booleans

Strictly holds **true** or **false** values for evaluation logic.

Undefined vs. Null — The Distinction

```
let city; // JS automatically assigns undefined
console.log(city); // Output: undefined

let selectedCountry = null; // Developer explicitly clears value
console.log(selectedCountry); // Output: null
```

undefined

Variable is declared, but **no value was assigned** yet. JS engine sets this automatically.

null

Intentional emptiness. Developer explicitly assigns null to mean "currently no value".

Symbol & BigInt Brief Overview

```
// Symbol – Creates guaranteed unique identifiers  
let uniqueId = Symbol("id");  
console.log(uniqueId);  
  
// BigInt – Handles arbitrarily large integers  
let bigNumber = 123456789012345678901234567890n;  
console.log(bigNumber);
```

Symbol

- Guarantees complete uniqueness
- Used for private object properties

BigInt

- Handles numbers beyond safe integer limits
- Append 'n' to literal numbers

Non-Primitive / Reference Types

// Object

```
let student = { name: "Sara", age: 21 };
```

// Array

```
let colors = ["red", "green", "blue"];
```

// Function

```
function greet() { console.log("Hello!"); }
```

Multiple Values

Hold collections of data points rather than single values.

Stored by Reference

Stored in heap memory. Variables point to memory addresses, not direct values.

The typeof Operator & Famous JS Quirk

```
typeof "Hello"; // "string"  
typeof 25; // "number"  
typeof true; // "boolean"  
typeof undefined; // "undefined"  
typeof null; // "object" ⚠  
typeof {}; // "object"
```

🚨 The `typeof null` Bug

Legacy bug from 1995! **typeof null** returns **"object"** due to how values were stored in 32-bit units. It cannot be fixed now as it would break legacy websites.

Explicit Type Conversion

```
// To String
String(100); // "100"

// To Number
Number("50"); // 50
Number("Hello"); // NaN (Not a Number)
Number(true); // 1

// To Boolean
Boolean(1); // true
```

Intentional Casting

Developer manually converts one type to another using built-in functions.

What is NaN?

Not a Number. Represents an invalid numeric conversion result.

The 6 Falsy Values in JavaScript

Only these **6 values** evaluate to **false** when converted to Boolean. Everything else is **truthy**!

0

NumberZero

""

Empty String

null

Null Value

undefined

Undefined

NaN

Not a Number

false

Boolean False

Type Coercion (Automatic Conversion)

```
// String coercion with +  
console.log("5" + 3); // "53"  
  
// Numeric coercion with -, *, /  
console.log("5" - 3); // 2  
console.log("5" * "2"); // 10  
console.log("10" / "2"); // 5
```

+ Plus Operator (+)

Converts numbers to strings when paired with a string.

Arithmetic (-, *, /)

Converts numeric strings into actual numbers automatically.

Loose (==) vs. Strict (===) Equality

Operator	Name	Type Coercion?	Example	Result
==	Loose Equality	Yes (Converts types first)	5 == "5"	true
===	Strict Equality	No (Checks value AND type)	5 === "5"	false



Golden Rule of Equality

Always use strict equality (===) to prevent unpredictable coercion bugs!

JavaScript Operators Overview

Arithmetic

+, -, *, /, % (Remainder), **
(Exponent)

Assignment

=, +=, -=, *=, /=

Comparison

>, <, >=, <=, !=, ===

Logical

&& (AND), || (OR), ! (NOT)

Interactive Practice Task

Task Checklist

- ☒ Declare 5 variables: string, number, boolean, undefined, null
- ☒ Use typeof on each variable and log output to console
- ☒ Test `"10" + 5` vs `"10" - 5` (Guess output first!)
- ☒ Write example comparing `==` vs `===`
- ☒ Bonus: Write all 6 falsy values from memory

Recap & Homework Assignment

✓ Key Takeaways

- 7 Primitives vs Non-Primitives
- `typeof null` returns "object"
- Explicit conversion vs implicit coercion
- Memorize 6 falsy values
- Always use `===` over `==`

🏠 Homework

- Complete practice checklist in VS Code.
- Write 5 custom comparisons testing `==` vs `===` with edge cases.

UP NEXT: DAY 03

Conditional Statements

if/else, switch statements, and ternary operators — where our boolean logic & comparison operators come to life!

Thank You! Allah Hafiz